Library Management System - Design Patterns

1. Repository Pattern

Type: Structural **Location in Code:** auth-service.repository.UserRepository, catalog-service.repository.BookRepository **Description:** Responsible for CRUD operations for entities independent of business logic. **Code Example:**

```
public interface UserRepository extends JpaRepository<AppUser, UUID> {
    Optional<AppUser> findByEmail(String email);
}
```

---

2. DTO Pattern

Type: Structural **Location in Code:** catalog-service.dto.BookDTO, auth-service.dto.UserDTO **Description:** Separates domain entities from API responses for safer and cleaner data transfer. **Code Example:**

```
@Data
@Builder
public class BookDTO {
    private String title;
    private String author;
    private String isbn;
}
```

---

3. Dependency Injection

Type: Behavioral **Location in Code:** AuthService, BookService, BorrowService **Description:** Spring @Service and @Autowired for decoupling and testability. **Code Example:**

```
@Service
public class AuthService {
    private final UserRepository userRepository;

    public AuthService(UserRepository userRepository) {
        this.userRepository = userRepository;
```

```
    }
}
```

---

4. Specification Pattern

Type: Behavioral **Location in Code:** BookSpecifications.java **Description:** Dynamically generate JPA Criteria queries for searching books. **Code Example:**

```
public class BookSpecifications {
    public static Specification<Book> hasTitle(String title) {
        return (root, query, cb) -> cb.like(root.get("title"), "%" + title +
"%");
    }
}
```

---

5. API Gateway Pattern

Type: Architectural **Location in Code:** api-gateway **Description:** Central entry point for frontend to route requests to microservices. **Code Example:**

```
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: https://library-auth-service.onrender.com
          predicates:
            - Path=/api/auth/**
```

---

6. Service Discovery Pattern

Type: Architectural **Location in Code:** discovery-server, @EnableEurekaServer, clients **Description:** Services register themselves in Eureka; gateway locates them by name. **Code Example:**

```
@EnableEurekaServer
@SpringBootApplication
public class DiscoveryServerApplication {}
```

7. AOP / Proxy Pattern

Type: Behavioral **Location in Code:** LoggingAspect.java **Description:** Centralized request/response logging for all services. **Code Example:**

```
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.library..*Service.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        log.info("Entering method: " + joinPoint.getSignature().getName());
    }
}
```

---

8. Builder / Factory Pattern

Type: Creational **Location in Code:** DTO.builder() in common-lib, factories **Description:** Create objects safely and consistently, especially DTOs or default values. **Code Example:**

```
BookDTO book = BookDTO.builder()
    .title("Clean Code")
    .author("Robert C. Martin")
    .isbn("9780132350884")
    .build();
```